Enhancing Textual Relatedness Assessment with Combined Transformers-Embedding Similarity Techniques and Machine Learning Regressors.

1st Mena Hany
Faculty of Computer Science
University for Modern Sciences and Arts
Cairo, Egypt
mena.hany@msa.edu.eg

2nd Mostafa Mohamed Saeed
Faculty of Computer Science
University for Modern Sciences and Arts
Cairo, Egypt
mmsaeed@msa.edu.eg

3rd Rana Reda Waly
Faculty of Computer Science
University for Modern Sciences and Arts
Cairo, Egypt
raelsayed@msa.edu.eg

4th Abdelrahman Ezzeldin Nagib
Faculty of Computer Science
University for Modern Sciences and Arts
Cairo, Egypt
abezzeldin@msa.edu.eg

5th Wael H. Gomaa
Faculty of Computer and Artificial Intelligence, Beni-Suef University
Faculty of Computer Science, MSA University
Egypt
Wahassan@msa.edu.eg

***Abstract*—In natural language processing (NLP), accurately calculating relatedness scores between pairs of sentences has significant implications for various tasks, such as information retrieval and document clustering. To improve relatedness score calculation, this study employed a two-layered approach. Firstly, embedding similarity techniques were utilized, leveraging seven different transformers to obtain vectors for each pair of sentences. Secondly, a classical machine learning regressor was trained on these seven vectors. The combination of these methods resulted in state-of-the-art performance on a standard dataset named SICK, with a mean square error of 0.0481, and Pearson's and Spearman's correlations of 0.978 and 0.9696, respectively. This research highlights the potential of combining embedding similarity techniques with machine learning methods to enhance relatedness score assessment and other NLP tasks, ultimately contributing to the advancement of NLP algorithms and their applications.**

***Index Terms*—Text Relatedness, Text Similarity, Transformers, Embeddings, Deep Learning, Machine Learning**

I. Introduction

Natural Language Processing (NLP) is a field that focuses on teaching computers to comprehend, use, and produce human language. It is used in various applications, including chatbots and systems that can perform relatedness score detection. NLP can help automate daily tasks due to the wide range of applications that can benefit from it. relatedness score detection is one of these applications, relatedness score detection is the ability to determine whether two sentences have near semantic meaning or not. One of these applications is relatedness score detection, which can assess whether or not two sentences have near semantic meaning. One may consider relatedness score detection to be a crucial NLP task that is involved in many more challenging NLP tasks including information retrieval, text mining, and semantic search. Applying relatedness score detection in NLP may appear to be easy work at first, but it is actually a challenging effort because of the complexity of the language and since many prior studies had a difficult time determining whether two phrases were actually related to one another or not. When professors and teachers are grading their students' research or assignments, for instance, they may want to know whether the research or assignment was plagiarised or not by checking to see if the students had copied from one another or from an online source. The most important step in the relatedness score detection procedure is the recognition of related phrases. NLP can be used in this situation. The development of a relatedness score detection system will mix NLP and Machine Learning (ML) techniques. The detection of relatedness between two sentences will be the main goal of this effort. To determine how closely two sentences resemble one another, a lot of NLP approaches can be typically applied. Later on in this research, it will be talked about the different methods for relatedness score detection. Included in this is our method for determining if two sentences are related to one another by calculating the similarity scores of the sentences and using a ML model.

A review of previous research on the topic and background information will be provided in the second section. In the third section, the project's methodology and system architecture will be discussed. The fourth section will discuss the data that will be utilized in the study, including the setting, equipment, and data, along with how the experiments were organized, carried out and their results. The fifth portion concludes the system's whole body of work and describes the forthcoming works. In the sixth segment, a discussion of the outcomes will be made.

979-8-3503-3556-9/23/$31.00 ©2023 IEEE

II. Related works

The researchers Pawar and Mago [13] introduced a model that has two steps to get the relatedness score of the two sentences. The first step is to input the two sentences to the first process that was called "maximize the similarity" that has three steps inside it that are to get the word similarity firstly, secondly is to get the semantic similarity and lastly is to get the word order similarity. After getting those similarity scores, they are fed to the second step of their model that was called "Bound the Similarity". This step also has three processes to get the relatedness score between the two sentences. The first one is called "Recurrence of Words", the second one is called "Negation and Stanford POS Tagger Model" [9] and the last model that was called "Spacy's Dependency Parser Model" and this last model used the dependency parsing form the library called spacy [4] and this model exceeded the previous two models.

Ichida et al. [5] introduced an approach that has three processes. the first process is to convert the sentences to a numeric vector representation using the word2vec skip-gram pre-trained model to produce this representation. So, it can be fed to the second process which is the Siamese neural network that the researchers tried two different types of hidden layers. Those hidden layers are the LSTM and GRU layers of the Recurrent neural network and the last step in their model is to measure the distance between the outputs of the Siamese neural network with the Manhattan Distance algorithm. After they examined the results the Siamese neural network with the GRU layers outperformed the LSTM in the results.

Liu et al. [8] came up with a model that has three methods. the first method is to convert the sentences to vectors using GloVe pre-trained model to produce a 300-dimensional vector and the next method is to input the vectors to the Stanford Neural Network Dependency Parser to produce the dependency trees that will then be fed to the last method that is the TreeLSTM model to produce the relatedness score between the two sentences.

Agarwala et al. [1] came up with a new approach that was called Novel Model that has two steps. The first step is to input the sentences to the pre-trained model called universal sentence encoder that generates a 500-dimensional vector and the second step is two input those vectors to a neural network that has a lambda layer, concatenate layer to concatenate the two embeddings, batch normalization layer, Dense layer, and Dropout layer to avoid over-fitting on the test data and lastly output layer to make the relatedness score output.

Li et al. [7] presented an approach that has five steps. The first step is to input the sentences into two pre-trained models to output numeric vectors. One of the models is the GloVe pre-trained model that produces a 300-dimensional vector and the other model is the PARAGRAM that produces a 25-dimensional vector. the next step is to combine the two vectors to produce a 325-dimensional vector that is fed to multiple Bi-LSTM hidden layers and after that to a Soft-alignment attention layer and finally to fully connected layers to make the final output.

Babic et al. [2] presented a model that has two processes. The first step is to produce the numeric vectors using the word2vec pre-trained model on the UMBC corpus and also using a word2vec pre-trained model on a combination of the UMBC corpus and the NASARI dataset and this pre-trained model. They called it the NASARI+word2vec model. After they produced those numeric vectors, the vectors were fed to five different methods for measuring the text semantic similarity. the first method was called sim_{cos}, the second method was called sim_{cos2}, the third method was called sts, the fourth method was called sts_s and the fifth and final method was called sts_{s2}. After examining those different methods and the different pre-trained models. they found that the combination of the sim_{cos2} method with the word2vec model outpreformed the other experiments.

Mueller and Thyagarajan [11] introduced a model that has two processes. The first process is to convert the sentences into vector representation by using the LSTM neural network

TABLE I
PREVIOUS WORKS

previous work	Pearson's correlation	Spearman's correlation	Mean Square Error
Recurrence of Words - Negation and Stanford POS Tagger Model - Spacy's Dependency Parser Model [13]	0.5878 - 0.6645 - 0.8324	0.5147 - 0.5964 - 0.6854	0.5585 - 0.4389 - 0.3784
Siamese LSTM model - Siamese GRU model [5]	0.7983 - 0.8448	0.7492 - 0.7902	0.3779 - 0.3032
Liu et al. [8]	0.871	0.834	-
Novel Model [1]	0.7722	0.6898	-
li et al. [7]	0.8993	0.8151	0.2529
sim_{cos2} method with the word2vec model [2]	0.589	0.519	-
MaLSTM [11]	0.8822	0.8345	0.2286
Constituency Tree-LSTM - Dependency Tree-LSTM [18]	0.8582 - 0.8676	0.7966 - 0.8083	0.2734 - 0.2532
Lai and Hockenmaier [6]	0.802	-	-
Bjerva et al. [3]	0.842	-	0.297

and this neural network is not to make predictions but it is to output a vector representation only. The second step is to take the output from the LSTM neural network and input it to the Manhattan Distance algorithm to produce the relatedness score output. They called this model MaLSTM.

Tai et al. [18] produced a model that has two methods. the first method is to take the sentences and input them into a TreeLSTM model to produce the similarity scores between the sentences. The second method is to take the similarity scores and input them to the Constituency Tree-LSTM model and the Dependency Tree-LSTM model. After examining those two models, it is deduced that the Dependency Tree-LSTM model achieved better results than the Constituency Tree-LSTM model and also they noted that the tree-LSTM models can outperform the sequential LSTM models.

Lai and Hockenmaier [6] produced a model that has three steps. The first step is to preprocess the sentences with lemmatization with the Stanford CoreNLP system, the Malt parser to identify five sets of constituents, the Illinois Chunker to extract syntactic chunks, and lastly the NLTK library to remove the stopwords. The second step is to extract different features like Distributional constituent similarity, Chunk label alignment and matching, alignment, Negation, Denotational constituent similarity, Word overlap, Unaligned chunk matching, Synonyms, and Hypernyms and Antonyms. The last step is to input the features to a log-linear regression model used from the open-source program called weka. They tried different combinations of the features but the best combination is to input all the features together into the log-linear regression model.

Bjerva et al. [3] produced a model that has two methods. The first method is to extract features from the sentences like RTE, Compositional Distributional Semantic Model, Wordnet novelty, Discourse Representation Structure, Logical model, Synset Overlap, Synset Distance, IDs, Sentence Lengths, and Word overlap. After extracting those features, They are inputted into a Random Forest Regressor to make the relatedness score output.

III. Methodology

A. Overview

First, pair of sentences will be read. The second step and the most crucial part is to extract the features that will be used. There are many forms of feature extraction techniques like the ones that were mentioned in the Related Works part. After examining the different approaches of different researchers, it is deduced that embedding similarity techniques can outperform other feature extraction techniques and the number of those embedding techniques will be 7 pre-trained models. The embedding similarity technique [17] is that a pre-trained model is used to transform the sentences to a numeric vector representation and after that, some similarity technique like the cosine similarity or any other similar techniques is used to take the two vectors. The output of the 7 models will be 14 different vectors for each sentence pair and compare how similar every two corresponding vectors are to each other to get a total of 7 similarity scores for each sentence pair. The final step is to input those similarity scores to a ML Regressor to predict the relatedness between the two sentences. Those three steps of the model are shown in fig 1

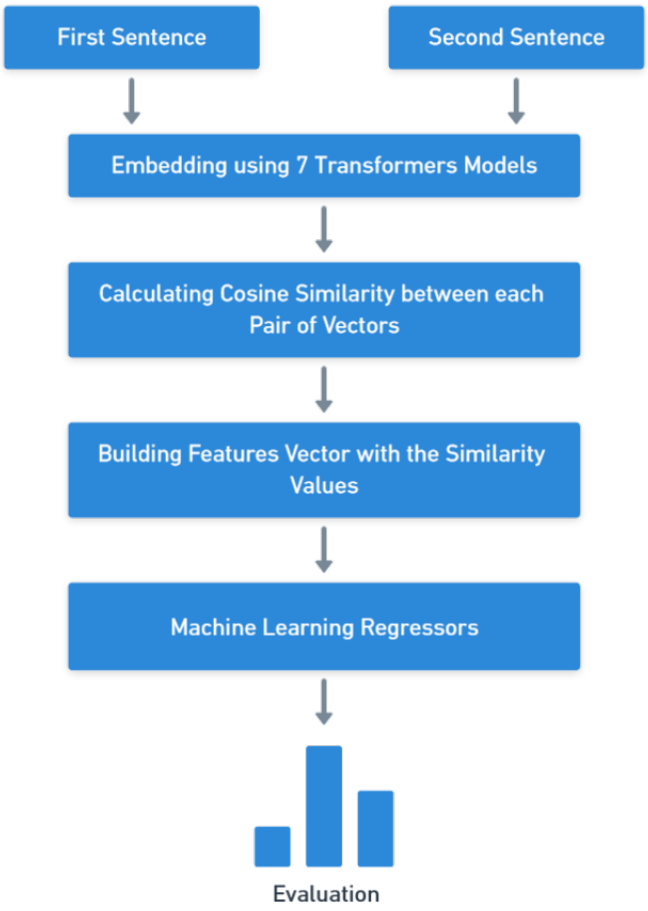

Fig. 1. System Overview

B. Data-set

the SICK (Sentences Involving Compositional Knowledge) dataset was originally built from two datasets. The first one is the 8K ImageFlickr dataset and the second one is the SemEval 2012 STS MSR-Video Description dataset. However, the dataset is not created only from the concatenation of those two datasets. The dataset went through 3 steps to reach this finalized version of the dataset. Firstly the pairs of sentences were normalized to remove or clarify sentences that have some lexical or semantic occurrences that are not correct to be used in the CDSMs (Compositional Distributional Semantic Models). Secondly, the sentence pairs were expanded through many different ways to make the one sentence pair create three other sentence pairs. Finally, the produced sentences from the sentence normalization and sentence expansion steps were concatenated with each other to create the final output of the dataset and this resulted in a total of 9840 sentences. The objective of the dataset is to predict the relatedness score that ranges from 1 to 5 [10].

TABLE II
DIFFERENT MODELS RESULTS

Model name	Pearson's correlation	Spearman's correlation	Mean Square Error
RandomSubSpace	**0.978**	**0.9696**	**0.04809249**
RandomForest	0.8969	0.8404	0.20241
KStar	0.8944	0.8371	0.207662
EnsembleSelection	0.893	0.8357	0.209489
Bagging	0.8925	0.8339	0.210589
AlternatingModelTree	0.8903	0.8345	0.214554
MLPRegressor	0.8872	0.8291	0.220242
RBFRegressor	0.8862	0.8289	0.222029
RandomCommittee	0.8846	0.8228	0.225815
REPTree	0.8828	0.8227	0.22858
DecisionTable	0.8797	0.8165	0.233966
PaceRegression	0.874	0.8201	0.244234
MultiSearch	0.874	0.8201	0.244234
LinearRegression	0.8739	0.82	0.244332
SMOreg	0.8739	0.8204	0.2496
LeastMedSq	0.8736	0.8195	0.247009
MultilayerPerceptron	0.8722	0.808	0.252406
MultilayerPerceptronCS	0.8722	0.808	0.252406
ElasticNet	0.8626	0.7985	0.269984

C. Evaluation metrics

In regression problems, there are a lot of evaluation metrics however the most commonly used metrics are Pearson's correlation, Spearman's correlation, and the Mean Square Error (MSE). A measurement of how strongly two variables are linearly associated is provided by the Pearson correlation coefficient [16]. The strength and direction of the arbitrary monotonic link between two ranked variables, or between one ranking variable and one measurement variable, are measured by the Spearman correlation coefficient, a nonparametric or distribution-free rank statistical measure [19]. In addition to the variance (precision) of the estimator, which shows how much it fluctuates about its expected value owing to sampling variability, MSE also displays the bias (accuracy) of the estimator, which is how much its expected value deviates systematically from the true value [15]. Pearson's correlation and Spearman's correlation when they approach 1 means that the model is doing very well and when they approach 0 means the model is fitting poorly on the data. While the MSE is vice versa from Pearson's correlation and Spearman's correlation meaning that when the MSE approaches small values near 0 means, as this is the lower limit of the MSE, the model is doing very well and when it is greater than 1 or more as the MSE has no upper limit like Pearson's correlation and Spearman's correlation means that it is fitting poorly on the data.

IV. RESULTS AND EXPERIMENTS

The experiments were done on the SICK dataset with 10-fold k-fold cross validation, with 16 GB ram, a processor Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz, and with a GPU NVIDIA GTX 1660ti 6GB.

A. Experiments

The experiments were done with a combination of 7 pre-trained models to transform the sentences into numeric

TABLE III
PRE-TRAINED MODELS DATA

Model Name	Output vector dimensions	Trained with
sentence-t5-large	768	2B question-answer pairs from diverse online communities.
distiluse-base-multilingual-cased	512	Multi-Lingual model of Universal Sentence Encoder for 15 languages
multi-qa-mpnet-base-dot-v1	768	215M (question, answer) pairs from diverse sources.
multi-qa-distilbert-cos-v1	768	215M (question, answer) pairs from diverse sources.
paraphrase-xlm-r-multilingual-v1	768	trained on data for 50+ languages
paraphrase-multilingual-MiniLM-L12-v2	384	trained on data for 50+ languages
multi-qa-MiniLM-L6-cos-v1	384	215M (question, answer) pairs from diverse sources.

vectors. The 7 models are sentence-t5-large, distiluse-base-multilingual-cased, multi-qa-mpnet-base-dot-v1, multi-qa-distilbert-cos-v1, paraphrase-xlm-r-multilingual-v1, paraphrase-multilingual-MiniLM-L12-v2 and multi-qa-MiniLM-L6-cos-v1. The different pre-trained model's output vectors and the data that they were trained on [12] [14]. At the table III. The 7 models will output 14 vectors for each pair of sentences.

B. Similarity techniques

The vectors are inputted into the cosine similarity technique to get the similarity score. A number between -1 and 1 is returned by the cosine similarity function, which computes the cosine of the angle between two vectors. A value of 1 indicates that the vectors are identical, a value of 0 shows that they are orthogonal, and a value of -1 indicates that they are entirely different. Because it is effective and can handle high-dimensional data, cosine similarity is a widely used approach. It is effective in applications where the size of the data changes significantly since it is resilient to differences in magnitude between vectors. The 14 vectors outputted from the pre-trained models will be fed to cosine similarity to get a 7 similarity score for each sentence pair.

C. Machine Learning models

The similarity scores are inputted into different machine-learning techniques as shown in table II and the data in table II are in descending order according to Pearson's correlation. This is to get the best machine learning techniques. RandomSubSpace regressor With a mean square error of 0.0481, a Pearson's correlation of 0.978, and Spearman's correlation of 0.9696. RandomSubSpace is an ensemble technique for feature selection and regularisation in machine learning.

V. Discussion

For predicting relatedness scores between pairs of sentences, a combination of seven Sentence Transformers models and a RandomSubSpace regressor has been shown to be effective. A wide range of semantic and contextual variables from the input text can be captured by the model thanks to the usage of numerous language models that have been trained on various datasets and architectural frameworks. These features can be crucial for accurately predicting relatedness scores. However, the RandomSubSpace regressor also contributes significantly to this combination's ability to produce the best outcomes. Popular ensemble techniques for feature selection and regularisation in machine learning include the RandomSubSpace algorithm. It operates by picking a subset of the input features at random, applying a regression model to each one, and then averaging the results of all the models to provide the final result. In high-dimensional datasets, where feature selection can be difficult and overfitting is a worry, this strategy can be extremely helpful. The RandomSubSpace regressor enhances the model's generalization capabilities and assists in preventing overfitting in the context of relatedness score prediction. The model is not unduly dependent on any single feature or collection of features, which can be a typical problem in natural language processing jobs, thanks to the algorithm's capability to select a subset of the most relevant features for each regression model. The final output of several regression models trained on various feature subsets is less prone to noise and variability in the input data since it averages the predictions from those models. An effective method for predicting relatedness scores is the combination of seven Sentence Transformers models and a RandomSubSpace regressor. The RandomSubSpace regressor guarantees the model is robust, generalizes well to unseen data, and achieves the best level of accuracy while the Sentence Transformers models collect

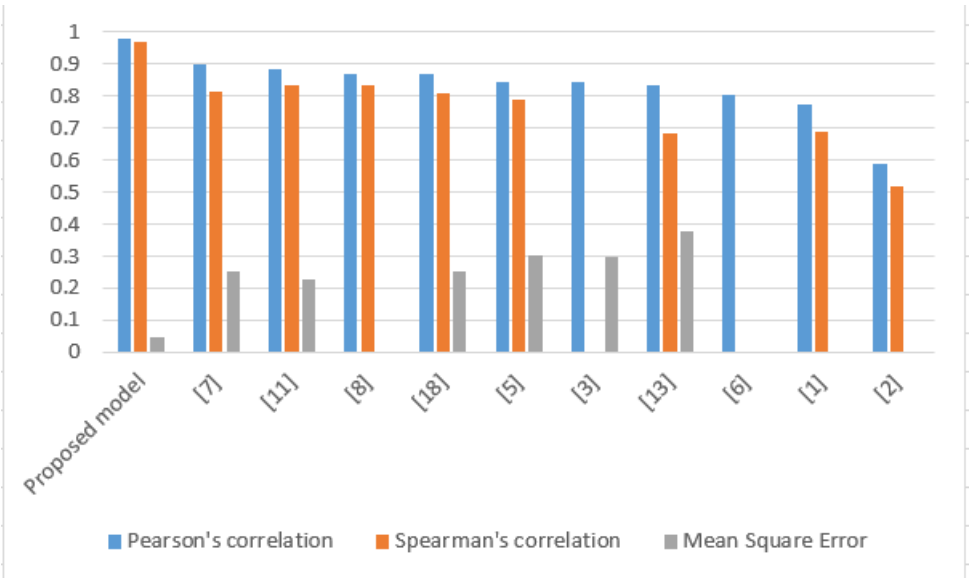

Fig. 2. Proposed model compared with previous works

a wide range of semantic and contextual characteristics from the input text.

VI. Conclusion

To sum up, the seven Sentence Transformers models in combination with the RandomSubSpace regressor have demonstrated promising results in properly predicting relatedness scores between sentence pairs. a mean square error of 0.0481, a Pearson's correlation of 0.978, and a Spearman's correlation of 0.9696, and those results can be seen in fig 2. The fig 2 compares the proposed model results with the highest results of the previous related works and some of the works have no results in the Spearman's correlation or the Mean Square Error. So, they have no bars in them. The model is able to capture a varied range of semantic and contextual characteristics from the input text by utilizing numerous language models trained on various datasets and architectures, leading to a more accurate and reliable output. The RandomSubSpace regressor additionally enables effective feature selection and regularisation, preventing overfitting and guaranteeing the model generalizes well to unobserved data.

Overall, this set of models provides a strong answer for a range of NLP applications, including document clustering, semantic search, and information retrieval. The efficiency and effectiveness of these applications can be considerably enhanced by the model's capacity to precisely capture the content and context of text documents, producing more accurate and pertinent outputs. We should expect to see increasingly complex and potent models emerge as the science of natural language processing develops, greatly enhancing our capacity to analyze and comprehend human language.

References

[1] S. Agarwala, A. Anagawadi, and R. M. R. Guddeti, "Detecting semantic similarity of documents using natural language processing," *Procedia Computer Science*, vol. 189, pp. 128–135, 2021.

[2] K. Babić, F. Guerra, S. Martinčić-Ipšić, and A. Meštrović, "A comparison of approaches for measuring the semantic similarity of short texts based on word embeddings," *J. Inf. Organ. Sci.*, vol. 44, no. 2, pp. 231–246, Dec. 2020.

[3] J. Bjerva, J. Bos, R. Van der Goot, and M. Nissim, "The meaning factory: Formal semantics for recognizing textual entailment and determining semantic similarity," in *Proceedings of the 8th International Workshop on Semantic Evaluation (SemEval 2014)*, 2014, pp. 642–646.

[4] M. Honnibal, "Spacy (version 1.3. 0)," *Spacy Web site (https://spacy.io/)*, 2016.

[5] A. Y. Ichida, F. Meneguzzi, and D. D. Ruiz, "Measuring semantic similarity between sentences using a siamese neural network," in *2018 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2018, pp. 1–7.

[6] A. Lai and J. Hockenmaier, "Illinois-lh: A denotational and distributional approach to semantics." in *SemEval@ COLING*, 2014, pp. 329–334.

[7] X. Li, C. Yao, Q. Zhang, and G. Zhang, "Semantic similarity modeling based on multi-granularity interaction matching," *International Journal of Innovative Computing, Information and Control*, vol. 15, no. 5, pp. 1685–1700, 2019.

[8] L. Liu, W. Yang, J. Rao, R. Tang, and J. Lin, "Incorporating contextual and syntactic structures improves semantic similarity modeling," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 1204–1209. [Online]. Available: https://aclanthology.org/D19-1114

[9] C. D. Manning, M. Surdeanu, J. Bauer, J. R. Finkel, S. Bethard, and D. McClosky, "The stanford corenlp natural language processing toolkit," in *Proceedings of 52nd annual meeting of the association for computational linguistics: system demonstrations*, 2014, pp. 55–60.

[10] M. Marelli, S. Menini, M. Baroni, L. Bentivogli, R. Bernardi, R. Zamparelli *et al.*, "A sick cure for the evaluation of compositional distributional semantic models." in *Lrec*. Reykjavik, 2014, pp. 216–223.

[11] J. Mueller and A. Thyagarajan, "Siamese recurrent architectures for learning sentence similarity," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 30, no. 1, 2016.

[12] J. Ni, G. H. Ábrego, N. Constant, J. Ma, K. B. Hall, D. Cer, and Y. Yang, "Sentence-t5: Scalable sentence encoders from pre-trained text-to-text models," *CoRR*, vol. abs/2108.08877, 2021. [Online]. Available: https://arxiv.org/abs/2108.08877

[13] A. Pawar and V. Mago, "Challenging the boundaries of unsupervised learning for semantic similarity," *IEEE Access*, vol. 7, pp. 16 291–16 308, 2019.

[14] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 11 2019. [Online]. Available: http://arxiv.org/abs/1908.10084

[15] M. D. Schluchter, "Mean square error," *Encyclopedia of Biostatistics*, vol. 5, 2005.

[16] P. Sedgwick, "Pearson's correlation coefficient," *Bmj*, vol. 345, 2012.

[17] H. Shahmohammadi, M. Dezfoulian, and M. Mansoorizadeh, "Paraphrase detection using lstm networks and handcrafted features," *Multimedia Tools and Applications*, vol. 80, pp. 6479–6492, 2021.

[18] K. S. Tai, R. Socher, and C. D. Manning, "Improved semantic representations from tree-structured long short-term memory networks," *arXiv preprint arXiv:1503.00075*, 2015.

[19] C. Xiao, J. Ye, R. M. Esteves, and C. Rong, "Using spearman's correlation coefficients for exploratory data analysis on big dataset," *Concurrency and Computation: Practice and Experience*, vol. 28, no. 14, pp. 3866–3878, 2016.